Workgroup:	Network Working Group
Internet-Draft:	draft-wullink-rpp-jscontact-profile-02
Published:	2 November 2026
Intended Status:	Standards Track
Expires:	6 May 2027
Authors:	M. Wullink	P. Kowalik
---	---	---
	SIDN Labs	DENIC

JSContact Profile for RESTful Provisioning Protocol (RPP)

Abstract

This document defines a JSContact usage profile, conforming to the rules described in RFC 10050, for the RESTful Provisioning Protocol (RPP). The JSContact Profile for RPP specifies how the JSContact format defined in RFC 9982 can be used as a standardized representation of contact information within RPP operations, enabling interoperability and consistency across RPP implementations that manage contact data.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 6 May 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (https://trustee.ietf.org/license-info) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction 3
1.1. Relationship to the RDAP JSContact Profile 3
2. Terminology 3
3. Conventions Used in This Document 4
4. Using JSContact in RPP 4
4.1. JSContact Profile for RPP 4
4.1.1. Version 4
4.1.2. Kind 4
4.1.3. Language 4
4.1.4. Name 4
4.1.5. Organizations 4
4.1.6. Addresses 4
4.1.7. Emails 5
4.1.8. Phones 5
4.1.9. Links 6
4.1.10. Map Keys 6
4.1.11. Localizations 6
4.1.12. Profile Properties 9
5. IANA Considerations 10
5.1. JSContact Profile Registry 10
6. Internationalization Considerations 10
7. Security Considerations 10
8. Change History 10
8.1. Version 01 to 02 10
8.2. Version 00 to 01 11
9. References 11
9.1. Normative References 11
9.2. Informative References 12

Acknowledgements 12

Authors' Addresses 12

1. Introduction

JSContact [RFC9982] is a flexible, JSON-based format for representing contact information. Its flexibility allows the same data to be expressed in multiple valid ways, which can hinder interoperability in a provisioning context where data must be stored and interpreted consistently.

This document defines a JSContact usage profile for the RESTful Provisioning Protocol (RPP), restricting the set of JSContact constructs that RPP clients and servers are required to support when provisioning contact information.

1.1. Relationship to the RDAP JSContact Profile

The Registration Data Access Protocol (RDAP) [RFC9083] provides read-only access to registration data managed by RPP. Both protocols use JSContact for contact representation, but with profiles suited to their respective roles.

The RDAP JSContact profile [I-D.ietf-regext-rdap-jscontact] defines the constructs used when serving registration data to read-only consumers. The RPP profile, defined in this document, is a subset of these constructs. All JSContact constructs permitted by the RPP profile can be used for RDAP.

The converse does not hold. RDAP supports additional JSContact constructs that are useful in the read-only context of RDAP responses but that may hinder interoperability. These constructs are not present in RPP requests and responses.

2. Terminology

In this document, the following terminology is used.

RESTful Provisioning Protocol or RPP - The protocol described in this document.

URL - A Uniform Resource Locator as defined in [RFC3986].

RPP client - An HTTP user agent performing an RPP request.

RPP server - An HTTP server responsible for processing requests and returning results in any supported media type.

Internationalized form - The representation of contact information in the Card members outside the `"localizations"` map.

Localized form - A representation of contact information placed in the `"localizations"` map, keyed by a language tag.

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

4. Using JSContact in RPP

4.1. JSContact Profile for RPP

Since JSContact is a generalized representation of contact data, many of its capabilities are not optimized for interoperable use in RPP payloads. This document therefore defines a usage profile for JSContact that simplifies implementation for both RPP clients and RPP servers.

The JSContact profile for RPP is compliant with the rules described in [RFC10050]. The profile properties are formally listed in Section 4.1.12. All types mentioned in this section refer to the JSContact specification [RFC9982].

4.1.1. Version

The Card `"version"` member value MUST be `"2.0"`.

4.1.2. Kind

If present, the Card `"kind"` member value MUST be `"individual"` (default) or `"org"` to represent an individual or an organization, respectively.

4.1.3. Language

The Card `"language"` member SHOULD be set when localizations are specified (i.e., when the `"localizations"` member is present and non-empty).

4.1.4. Name

Other than the optional `"@type"` member, the Name type MUST include only the `"full"` member.

4.1.5. Organizations

Other than the optional `"@type"` member, the Organization type MUST include only the `"name"` member.

4.1.6. Addresses

Other than the optional `"@type"` member, the Address type MUST include only the `"components"` and `"countryCode"` members.

Other than the optional `"@type"` member, the AddressComponent type MUST include only the `"kind"` and `"value"` members. The `"kind"` member value MUST be one of `"name"`, `"locality"`, `"region"`, or `"postcode"`.

The Address type MUST include the "countryCode" member. The "components" member MUST contain exactly one "locality" component, at most one "region" component, and at most one "postcode" component.

The "name" components represent street lines. The server MAY constrain the maximum number of "name" components supported. Their order in the "components" array is significant and MUST be preserved.

When both an internationalized and a localized version of an address are used, the localized version MUST be placed in the "localizations" map keyed by an appropriate [BCP47] language tag.

4.1.7. Emails

Other than the optional "@type" member, the EmailAddress type MUST include only the "address" member.

4.1.8. Phones

The Phone type MUST include the "number" member and MAY include the "features" member. If present, the "features" member MUST contain only the "voice" or "fax" keys, and the value associated with each key MUST be true. When the "features" member is absent, the phone number is assumed to be a voice number.

The "number" member value MUST be a "tel" URI [RFC3966] conforming to the rpp-telephone-uri rule in Figure 1. This rule restricts the [RFC3966] global-number to the telephone number format of the e164 type in Section 2.5 of [RFC5733]: a leading "+", a country code of one to three digits, a ".", and a subscriber number of up to 14 digits, optionally followed by an "ext" parameter carrying one or more digits. The total number of digits in the country code and subscriber number MUST NOT exceed 15. Local numbers, other visual separators, and other [RFC3966] parameters (e.g., "isub" and "phone-context") MUST NOT be used.

The grammar uses ABNF [RFC5234]. "DIGIT" is defined in [RFC5234].

```
rpp-telephone-uri        = "tel:" rpp-global-number
                           ; restricts telephone-uri [RFC3966]
rpp-global-number        = rpp-global-number-digits [ rpp-extension ]
                           ; restricts global-number [RFC3966]
rpp-global-number-digits = "+" 1*3DIGIT "." 1*14DIGIT
                           ; restricts global-number-digits [RFC3966]
rpp-extension            = ";ext=" 1*DIGIT
                           ; restricts extension [RFC3966]
```

Figure 1: ABNF for the Phone number member

Examples of valid values are "tel:+49.6215550100" and "tel:+49.6215550100;ext=123". An RPP server MUST reject a Phone whose "number" member value does not conform to this format.

4.1.9. Links

The Link type MUST include the "uri" member and MAY include the "kind" member. When the "kind" member is present, its value MUST be "contact".

4.1.10. Map Keys

A JSContact map key MUST comply with the Id type definition in [RFC9553]. RPP implementations MUST use the following predefined keys:

- "org" in the "organizations" map for a single preferred organization.
- "addr" in the "addresses" map for a single preferred postal address.
- "email" in the "emails" map for the preferred email address.
- "voice" in the "phones" map for the preferred voice number.
- "fax" in the "phones" map for the preferred fax number.
- "url" in the "links" map for a preferred contact URL; the "kind" member of the Link object MUST NOT be set.
- "contact-uri" in the "links" map for a contact URI as defined in [RFC8605]; the "kind" member of the Link object MUST be set to "contact".

When internationalized and localized versions of an organization, postal address, or email exist, the internationalized version MUST be included under the predefined map key, while the localized version MUST be placed in the "localizations" map as described in Section 4.1.11.

Keys MUST be of the form `"<predefined key>-<sequential number>"` for any additional entry in the `"emails"` or `"phones"` map, where the sequential number is a decimal integer starting at 1. For example, a `"phones"` map containing three voice numbers contains the keys `"voice"`, `"voice-1"`, and `"voice-2"`. An additional key MUST NOT be used unless the corresponding predefined key is also present, and the sequential numbers MUST NOT contain gaps.

4.1.11. Localizations

If present, localized variants of name, organization, postal address, and email MUST be added to the "localizations" map. RPP implementations MUST expand all localizations, meaning a nested PatchObject key of the form "{key1}/{key2}/.../{keyN}" MUST NOT be used.

The internationalized form MUST contain only characters that can be represented in the 7-bit US-ASCII character set [RFC20]. This applies to the `"full"` member of Name, the `"name"` member of Organization, the `"value"` member of AddressComponent, and the `"address"` member of EmailAddress. A localized form MAY contain any character permitted by [RFC9982]. When contact information is available only in a form containing non-ASCII characters, the RPP client MUST provide a US-ASCII transliteration as the internationalized form. An RPP server MUST reject a Card whose internationalized form contains characters outside the US-ASCII character set.

An RPP server MUST support at least one localization per Card. An RPP server MAY support more localizations, as determined by server policy. An RPP server MUST reject a Card that contains more localizations than it supports. The language of each localization MUST be indicated by a [BCP47] language tag as the key in the "localizations" map.

The following example of a Card includes two email addresses, two voice numbers, a fax number, and a localized version of the postal address:

```
{
  "@type": "Card",
  "version": "2.0",
  "language": "en",
  "name": {
    "full": "Joe User"
  },
  "organizations": {
    "org": { "name": "Example Org" }
  },
  "addresses": {
    "addr": {
      "components": [
        { "kind": "name",     "value": "Main Street 1" },
        { "kind": "locality", "value": "Ludwigshafen am Rhein" },
        { "kind": "region",   "value": "Rhineland-Palatinate" },
        { "kind": "postcode", "value": "67067" }
      ],
      "countryCode": "DE"
    }
  },
  "emails": {
    "email":   { "address": "joe.user@example.com" },
    "email-1": { "address": "joe@example.net" }
  },
  "phones": {
    "voice": {
      "features": { "voice": true },
      "number": "tel:+49.6215550100"
    },
    "voice-1": {
      "number": "tel:+49.6215550101;ext=1234"
    },
    "fax": {
      "features": { "fax": true },
      "number": "tel:+49.6215550199"
    }
  },
  "localizations": {
    "de": {
      "addresses": {
        "addr": {
          "components": [
            { "kind": "name",     "value": "Hauptstraße 1" },
            { "kind": "locality", "value": "Ludwigshafen am Rhein" },
            { "kind": "region",   "value": "Rheinland-Pfalz" },
            { "kind": "postcode", "value": "67067" }
          ],
          "countryCode": "DE"
        }
      }
    }
  }
}
```

Figure 2: Example of handling localizations in JSContact

4.1.12. Profile Properties

The following entries define the properties of the JSContact profile for RPP registered in the JSContact Profile registry as described in Section 5.1. The field names are those defined in [RFC10050]. Entry fields with empty values are omitted:

- Property Name: version
  Property Context: Card
- Property Name: language
  Property Context: Card
- Property Name: kind
  Property Context: Card
  Restricted Enum Values: individual, org
- Property Name: name
  Property Context: Card
- Property Name: full
  Property Context: Name
  Restricted Attributes: mandatory
- Property Name: organizations
  Property Context: Card
- Property Name: name
  Property Context: Organization
  Restricted Attributes: mandatory
- Property Name: addresses
  Property Context: Card
- Property Name: countryCode
  Property Context: Address
  Restricted Attributes: mandatory
- Property Name: components
  Property Context: Address
- Property Name: kind
  Property Context: AddressComponent
  Restricted Enum Values: name, locality, region, postcode
- Property Name: value
  Property Context: AddressComponent
- Property Name: emails
  Property Context: Card
- Property Name: address
  Property Context: EmailAddress
- Property Name: phones
  Property Context: Card
- Property Name: features
  Property Context: Phone
  Restricted Enum Values: voice, fax
- Property Name: number
  Property Context: Phone
- Property Name: links
  Property Context: Card

- Property Name: kind
  Property Context: Link
  Restricted Enum Values: contact
- Property Name: uri
  Property Context: Link
- Property Name: localizations
  Property Context: Card
  Restricted PatchObject: yes

The `"@type"` member is permitted on every JSContact object in this profile and is therefore not listed above.

An RPP server MUST reject a Card that contains a JSContact property not listed in this section.

5. IANA Considerations

5.1. JSContact Profile Registry

IANA is requested to register the following entry in the "JSContact Profile" registry [RFC10050]:

Name: rpp
Version: 1.0
Reference: Section 4.1.12 of this document

TODO

6. Internationalization Considerations

TODO

7. Security Considerations

RPP servers should impose appropriate limits on the size and complexity of JSContact objects received from RPP clients, including the number and size of expanded localizations.

TODO

8. Change History

RFC Editor: please remove this section before publication.

8.1. Version 01 to 02

- Replaced the reference to draft-ietf-calext-jscontact-profiles with RFC 10050
- Aligned with draft-ietf-regext-rdap-jscontact-27, making the RPP profile a subset of the RDAP JSContact profile, and made that reference informative (Issue #20)

- Made the Address `"countryCode"` member mandatory, removed Address `"full"` from the profile properties, limited the number of each AddressComponent kind, and made the order of street lines significant (Issue #19)
- Restricted the internationalized form to US-ASCII and required RPP servers to support at least one localization, with additional localizations subject to server policy (Issue #6)
- Allowed multiple email addresses and voice and fax numbers using sequentially numbered map keys (Issue #5)
- Required the Phone `"number"` member to be a `"tel"` URI (RFC 3966) restricted to the Extensible Provisioning Protocol (EPP) telephone number format, and added ABNF for it
- Added the terms internationalized form and localized form
- Permitted the `"@type"` member on every JSContact object and corrected the Organization member to `"name"`
- Made the Card `"kind"` member optional
- Converted the profile properties table into a list using the RFC 10050 field names
- Referenced RFC 8605 for the `"contact-uri"` map key
- Added the Card `"version"` member to the profile properties
- Required RPP servers, instead of client validation, to reject a Card containing properties outside the profile

8.2. Version 00 to 01

- Fixed duplicate text (Issue #14)

9. References

9.1. Normative References

[BCP47] Best Current Practice 47, <https://www.rfc-editor.org/info/bcp47>. At the time of writing, this BCP comprises the following:

Phillips, A., Ed. and M. Davis, Ed., "Matching of Language Tags", BCP 47, RFC 4647, DOI 10.17487/RFC4647, September 2006, <https://www.rfc-editor.org/info/rfc4647>.

Phillips, A., Ed. and M. Davis, Ed., "Tags for Identifying Languages", BCP 47, RFC 5646, DOI 10.17487/RFC5646, September 2009, <https://www.rfc-editor.org/info/rfc5646>.

[RFC10050] Stepanek, R. and M. Loffredo, "Protocol-Specific Profiles for JSContact", RFC 10050, DOI 10.17487/RFC10050, September 2026, <https://www.rfc-editor.org/info/rfc10050>.

[RFC20] Cerf, V., "ASCII format for network interchange", STD 80, RFC 20, DOI 10.17487/RFC20, October 1969, <https://www.rfc-editor.org/info/rfc20>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC3966] Schulzrinne, H., "The tel URI for Telephone Numbers", RFC 3966, DOI 10.17487/RFC3966, December 2004, <https://www.rfc-editor.org/info/rfc3966>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/info/rfc3986>.

[RFC5234] Crocker, D., Ed. and P. Overell, "Augmented BNF for Syntax Specifications: ABNF", STD 68, RFC 5234, DOI 10.17487/RFC5234, January 2008, <https://www.rfc-editor.org/info/rfc5234>.

[RFC9083] Hollenbeck, S. and A. Newton, "JSON Responses for the Registration Data Access Protocol (RDAP)", STD 95, RFC 9083, DOI 10.17487/RFC9083, June 2021, <https://www.rfc-editor.org/info/rfc9083>.

[RFC9553] Stepanek, R. and M. Loffredo, "JSContact: A JSON Representation of Contact Data", RFC 9553, DOI 10.17487/RFC9553, May 2024, <https://www.rfc-editor.org/info/rfc9553>.

[RFC9982] Stepanek, R., "JSContact Version 2.0: A JSON Representation of Contact Data", RFC 9982, DOI 10.17487/RFC9982, May 2026, <https://www.rfc-editor.org/info/rfc9982>.

9.2. Informative References

[I-D.ietf-regext-rdap-jscontact] Loffredo, M. and G. Brown, "Using JSContact in Registration Data Access Protocol (RDAP) JSON Responses", Work in Progress, Internet-Draft, draft-ietf-regext-rdap-jscontact-27, 5 October 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-regext-rdap-jscontact-27>.

[RFC5733] Hollenbeck, S., "Extensible Provisioning Protocol (EPP) Contact Mapping", STD 69, RFC 5733, DOI 10.17487/RFC5733, August 2009, <https://www.rfc-editor.org/info/rfc5733>.

[RFC8605] Hollenbeck, S. and R. Carney, "vCard Format Extensions: ICANN Extensions for the Registration Data Access Protocol (RDAP)", RFC 8605, DOI 10.17487/RFC8605, May 2019, <https://www.rfc-editor.org/info/rfc8605>.

Acknowledgements

TODO

Authors' Addresses

Maarten Wullink
SIDN Labs
Email: maarten.wullink@sidn.nl
URI: https://sidn.nl/

Pawel Kowalik
DENIC
Email: pawel.kowalik@denic.de
URI: https://denic.de/